LATS-Powered Debugging and Root Cause Analysis Agent

A complete technical implementation guide with pseudocode: Language Agent Tree Search built from scratch with a real LLM as policy, evaluator and reflector, and a real Python environment for execution

Audience	Python developers who know basic ML, LLMs and MCTS and want to implement LATS end to end
Stack	Python 3.10+, Anthropic API (any chat LLM works), subprocess sandbox, no frameworks
Deliverable	Architecture, math, pseudocode for every module, prompts, worked trace, tuning guide

Note on scope. This guide follows the project layout you described (main.py, node.py, environment.py, llm_client.py, policy.py, evaluator.py, reflection.py, reward.py, mcts.py, executor.py, three prompt files, test_debugger.py). All implementation details are given as language-neutral pseudocode that you can translate directly into Python; JSON formats and LLM prompts are given verbatim.

Contents

1. Project overview and design principles
2. System architecture
3. Environment, state and node design
4. The six LATS operations (selection, expansion, evaluation, simulation, backpropagation, reflection)
5. Reward design and objective evidence
6. Pseudocode with plain functions (no classes)
7. Prompts
8. Worked example: calculate_average([])
9. Budgets, depth limits, safety and failure handling
10. Testing, tuning and evaluation
11. Limitations and extensions

1. Project Overview and Design Principles

The agent receives (1) a Python function, (2) an error message or unexpected output, (3) a list of test cases, and (4) a description of expected behavior. Its job is to find the most likely **root cause**, not to blindly emit a patch. A plain LLM debugging prompt commits to one reasoning chain. If the first hypothesis is wrong (for example "the loop is broken" when the real problem is the missing empty-input contract), the chain rarely recovers. LATS replaces the single chain with a **search tree**: many candidate investigations are tried, real observations are collected, intermediate states are scored, and statistics are propagated so the search concentrates on promising branches while still exploring alternatives.

1.1 Why LATS fits debugging

- **Debugging is naturally a tree.** Every inspection or experiment can branch into several follow-ups.
- **Actions have real, checkable outcomes.** Running code gives ground truth, which LATS can use (environment feedback is a core ingredient of the algorithm).
- **Dead ends are informative.** LATS reflection turns a failed branch into a lesson that steers later branches.
- **Backtracking is free.** A node stores its full state, so the search can return to any earlier point without replaying history.

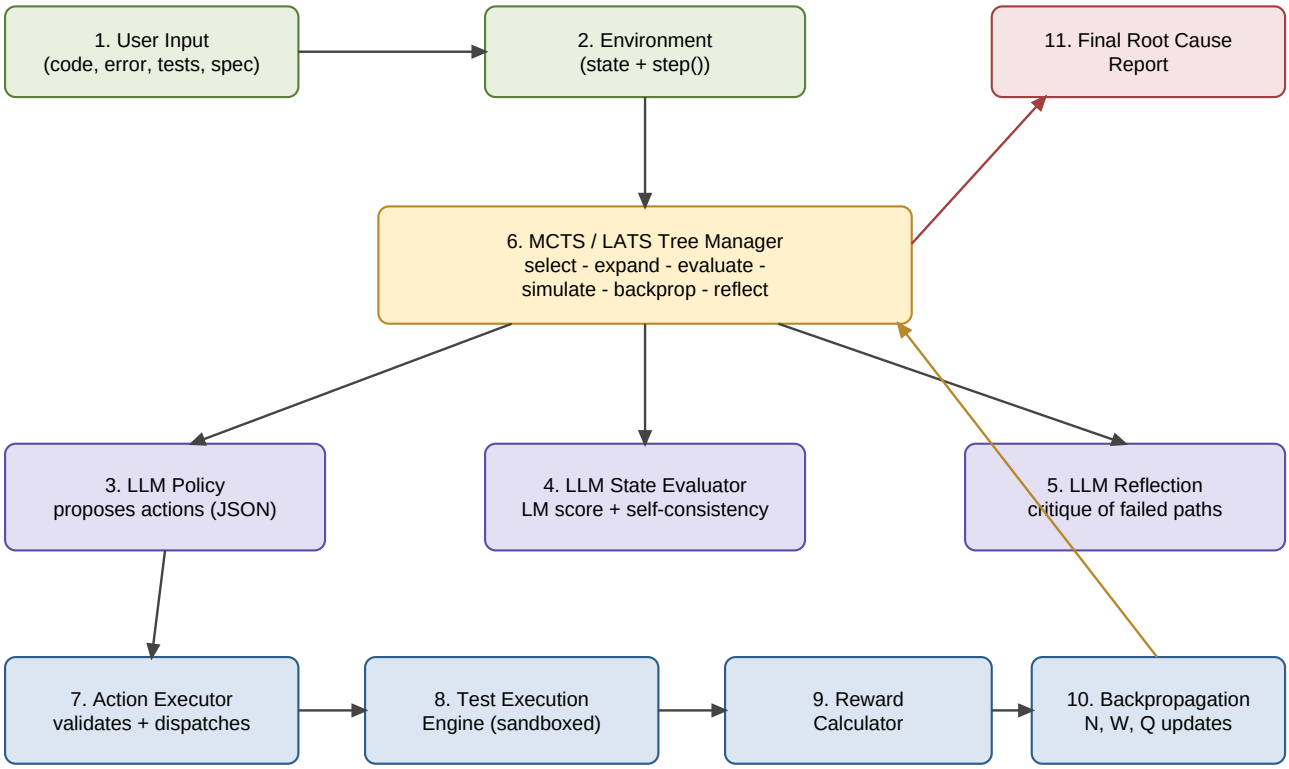
1.2 Hard requirements this guide enforces

- A real LLM generates actions, scores states and writes reflections. No rule-based debugging decisions.
- The LLM never produces test results. Every observation comes from the Python executor.
- Intermediate states are evaluated, not only final code.
- The final output is a root cause report whose claims are tied to recorded evidence.

1.3 Glossary

Term	Meaning in this project
State	Everything known at one point: code versions, error, test results, hypotheses, actions taken, evidence, summary.
Action	One structured debugging step chosen by the policy (inspect, run snippet, run tests, hypothesize, patch, conclude).
Observation	The real output from executing the action (stdout, stderr, test pass/fail, source excerpt).
Node	A search-tree vertex holding a state plus MCTS statistics, and the action/observation that led to it.
Trajectory	The ordered path root -> node: sequence of (action, observation) pairs.
Terminal state	A node where search stops along that branch: a conclusion was made, the depth limit hit, or the branch is invalid.

2. System Architecture



Real execution results flow Executor -> Test Engine -> Reward. The LLM never fabricates observations.

Figure 1. Component diagram. Purple = LLM-backed components, blue = deterministic Python components, yellow = search controller.

#	Component	Responsibility	LLM?
1	User Input	Provides code, error, tests, expected behavior. Parsed into the initial DebugState.	No
2	Environment	Owns the state schema; exposes step(state, action) -> (new_state, observation). Applies actions by calling the executor.	No
3	LLM Policy	Given a state (and prior reflections) proposes k diverse candidate actions as JSON.	Yes
4	LLM State Evaluator	Scores a state for progress toward the root cause. Sampled several times for self-consistency.	Yes
5	LLM Reflection Generator	Analyses failed or stalled trajectories and writes a lesson stored in memory.	Yes
6	MCTS Tree Manager	Runs select / expand / evaluate / simulate / backpropagate loop, enforces budgets.	No
7	Action Executor	Validates action JSON against a schema and dispatches to the right handler.	No
8	Test Execution Engine	Runs code and tests in an isolated subprocess with timeout; returns structured results.	No
9	Reward Calculator	Computes objective reward from environment evidence (tests, new facts, verified hypotheses, penalties).	No
10	Backpropagation	Updates N (visits) and W (value sum) up to the root; Q = W / N.	No

#	Component	Responsibility	LLM?
11	Final Report	Selects best conclusion and asks the LLM to write a report restricted to recorded evidence.	Yes (writer only)

2.1 Data flow for one iteration

- Controller **selects** a leaf by UCT from the root.
- Policy produces k actions for the leaf state. Executor validates and runs each; the environment returns a new state and observation, which become k child nodes.
- Each child is **evaluated**: V = beta * V_llm + (1 - beta) * R_obj, where V_llm = lambda * LM + (1 - lambda) * SC.
- The best child is **simulated** forward (greedy rollout, real execution) to a terminal or depth limit.
- The terminal value is **backpropagated** along the path. If the rollout failed, a **reflection** is generated and stored.

3. Environment, State and Node Design

3.1 What is a node here?

A node is one point in the investigation. It stores the **state** after executing the action on the edge from its parent, the **action** and **observation** that created it, plus search statistics. In this project **nodes are states** (the standard LATS formulation); the action lives on the incoming edge but is stored on the child for convenience.

Concept	Definition	Example
Node	Tree vertex: state + stats + links	Node #4 at depth 2
State	Full snapshot of knowledge at that node	Code, error, 3 test results, 1 hypothesis, 2 evidence items
Action	Structured step the agent chose	{"type":"run_snippet","code":"print(len([]))"}
Observation	Ground-truth result of the action	stdout: 0
Trajectory	Path of (action, observation) from root	inspect -> run snippet -> hypothesize
Terminal state	No further expansion on this branch	conclude action, depth limit, invalid/duplicate loop

3.2 Debug state

The state is a plain dictionary that can be deep-copied cheaply. Children copy the parent state and add to it; nothing is mutated in place, so backtracking is safe.

Field	Purpose
original_code	The code as given. Never changes.
current_code	Latest code version (changes only through apply_patch).
error_message	Original error or unexpected-output description.
test_cases	List of {call, expect} items.
expected_behavior	Natural-language contract.
test_results	Latest structured test results from the engine.
hypotheses	List of {id, text, status: open/supported/refuted, evidence_ids}.
actions_taken	Short strings for deduplication and for the policy prompt.
evidence	List of {id, kind, text} facts that came from real execution.
summary	One-paragraph running investigation summary (rebuilt from evidence, not free LLM prose).

3.3 Example search tree

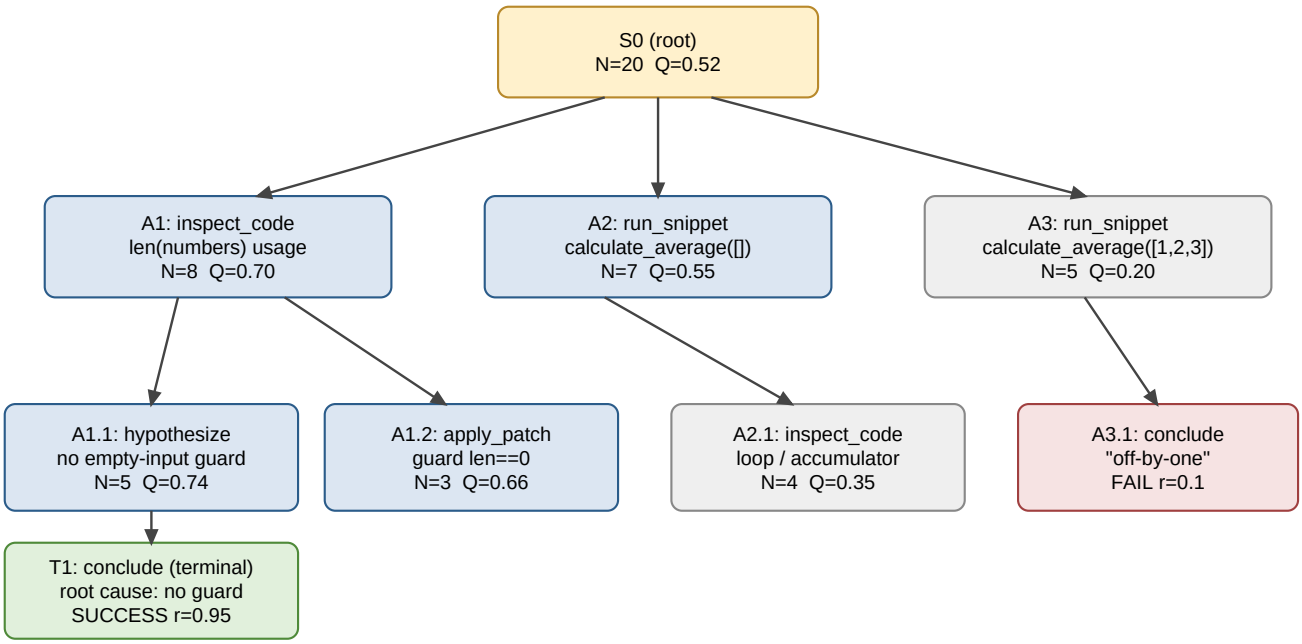


Figure 2. Search tree for calculate_average([]). Grey = low value branch, red = failed terminal conclusion, green = successful terminal. Q is average value, N is visit count.

Note that A3 (testing a non-empty list) was explored but yields little: the function works for normal input, so it remains available but with a low Q. A1 has the highest Q and also the most visits, so UCT will favor it unless a sibling's exploration bonus becomes larger.

4. The Six LATS Operations

LATS (Zhou et al., 2023, "Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models") adapts MCTS to language agents. The six operations are: **selection, expansion, evaluation, simulation, backpropagation, reflection**. The main loop:

```
for iteration in range(MAX_ITERATIONS):
    leaf      = select(root)                      # A. UCT descent
    if leaf.terminal: backprop(leaf, leaf.value); continue
    children  = expand(leaf)                      # B. LLM proposes, env executes
    for ch in children: ch.value = evaluate(ch)   # C. LM + SC + objective
    best      = argmax(children, key=value)
    end, v    = simulate(best)                    # D. real rollout
    backprop(end, v)                              # E. update N, W, Q on path
    if end.failed: reflect(end)                   # F. store lesson
return best_conclusion(root)
```

A. Selection

Selection walks from the root to a leaf, at every node choosing the child with the highest UCT (Upper Confidence bound applied to Trees):

`UCT(s,a) = Q(s,a) + c * sqrt( ln N(s) / N(s,a) )`

- **Q(s,a)**: average value of the child reached by action a from s (= W / N). This is the **exploitation** term: prefer branches that have scored well.
- **N(s)**: visit count of the parent. **N(s,a)**: visit count of the child. Their ratio drives the **exploration** term, which is large for rarely visited children and shrinks as a child is visited more.
- **c**: exploration constant. Large c explores more (values are in [0,1], so c between 0.5 and 1.4 is a sensible range; start at 1.0). Small c is greedy.
- Unvisited children (N(s,a) = 0) get +infinity so each action is tried at least once.
- ln N(s) grows slowly, so a neglected sibling's bonus rises every time the parent is visited and it eventually gets re-selected even if its Q is low.

Numerical example

Parent N(s) = 20, so ln 20 = 2.996. Four children:

Child	N(s,a)	Q	Bonus c=1.0	UCT c=1.0	Bonus c=0.3	UCT c=0.3
A inspect len usage	8	0.70	0.612	1.312	0.184	**0.884**
B run empty input	5	0.55	0.774	1.324	0.232	0.782
C check contract	2	0.40	1.224	**1.624**	0.367	0.767
D test negatives	4	0.30	0.865	1.165	0.260	0.560

With c = 1.0 the under-explored child C wins despite a mediocre Q: exploration dominates. With c = 0.3 child A wins: exploitation dominates. After C is visited a few more times its bonus falls (N=5 gives 0.774) and, if its Q stays low, selection returns to A or B. **An already explored branch is selected again** whenever its Q plus bonus exceeds all siblings; as N(s) grows, every sibling's bonus rises logarithmically, so no branch is permanently abandoned.

```
FUNCTION select(root, c):
    node <- root
    WHILE node has children AND node is not terminal:
        live <- children of node that are not pruned
        IF live is empty:
            mark node as pruned
            RETURN root            // restart descent; parent will skip this node
        node <- argmax over ch in live of UCT(node, ch, c)
    RETURN node
```

```
FUNCTION UCT(parent, child, c):
    IF child.visits = 0: RETURN +infinity
    RETURN child.average_value + c * sqrt( ln(max(parent.visits, 1)) / child.visits )
```

B. Expansion

At the selected leaf the LLM policy sees the state summary, the evidence list, the actions already tried and the stored reflections, and proposes **k** distinct actions (k = 3 to 5). The executor validates each, runs it against the real environment, and every successful action creates a child node holding the new state and the real observation.

Structured policy output

```
{
  "reasoning": "We know the crash is a ZeroDivisionError at total / len(numbers).",
  "actions": [
    {"type": "inspect_code",  "focus": "return statement", "rationale": "Locate the division"},
    {"type": "run_snippet",   "code": "print(len([]))",   "rationale": "Confirm denominator is 0 for []"},
    {"type": "run_snippet",   "code": "print(calculate_average([0]))", "rationale": "Distinguish zero-sum from empty"},
    {"type": "hypothesize",   "hypothesis": "No guard for empty input; contract undefined", "rationale": "Record leading idea"}
  ]
}
```

Action types

type	Required fields	What the environment does
inspect_code	focus	Returns numbered source lines matching a function name, line range or keyword (deterministic, no LLM).
run_snippet	code	Prepends current_code and runs the snippet in the sandbox; returns stdout/stderr/exit code.
run_tests	(none)	Runs all test_cases against current_code; returns pass/fail per test.
hypothesize	hypothesis	Records an open hypothesis. Observation is just "recorded": it adds no evidence by itself.
apply_patch	new_code, hypothesis_id	Replaces current_code in the child state and re-runs tests; used to test a hypothesis causally.
conclude	root_cause, evidence_ids	Terminal. Validated: must cite existing evidence ids.

Validation and invalid actions

- Reject JSON that fails to parse, has an unknown type, or lacks required fields.
- Reject duplicates: the action fingerprint (type + normalized payload) already exists in state.actions_taken or among siblings.
- Reject snippets over a length limit or containing forbidden imports (os, subprocess, socket, shutil, sys.exit) before execution.
- Reject conclude actions that cite no evidence ids or ids that do not exist.
- If every proposed action is invalid, retry the policy once with the rejection reasons appended; if still empty, mark the node pruned and apply a small penalty.
- A runtime error inside a snippet is **not** an invalid action: it is a valid observation (the error text becomes evidence).

Diversity. Ask the policy for actions of different categories (inspect, probe, hypothesize, test) and sample at temperature around 0.8. Reject near-duplicates by fingerprint. The prompt in section 7 explicitly instructs the LLM not to produce variations of the same probe.

C. Evaluation

Every new node is scored for how promising its **state** is, not merely whether code compiles. The evaluator considers: evidence collected, hypothesis quality, progress toward the root cause, correctness of observations, relevance of the investigation, and whether uncertainty has decreased. The LATS paper combines two signals:

```
V(s) = lambda * LM(s) + (1 - lambda) * SC(s)
```

- **LM(s)**: the LLM's direct numeric judgment of state quality, normalized to [0,1].
- **SC(s)**: self-consistency. Sample the evaluator (or the policy) m times at the same state and measure agreement. In this implementation SC = fraction of samples whose leading hypothesis id matches the modal one, so a state where the evaluator keeps reaching the same conclusion scores higher.
- **lambda**: weight between them. Start at 0.5; raise it if the evaluator is well calibrated, lower it if its scores are noisy.

Limitations of an LLM judging itself

- **Self-preference and overconfidence:** the model tends to rate its own hypotheses highly.
- **Sycophantic coherence:** a plausible-sounding story gets a high score even with no supporting evidence.
- **Variance:** scores shift with wording and temperature; self-consistency reduces but does not remove this.
- **Correlated errors:** sampling the same model repeatedly agrees with itself even when wrong.

Recommended practical implementation: hybrid value

```
V(s) = beta * [ lambda*LM(s) + (1-lambda)*SC(s) ]  +  (1 - beta) * R_obj(s)
```

R_obj comes from the environment (section 5): reproduced error, tests changed by a patch, hypotheses verified or refuted by experiments. Use beta = 0.5 to start. The evaluator prompt is also given the objective facts and told that unsupported claims must lower the score.

Evaluator output

```
{
  "score": 0.72,                        // 0.0 - 1.0
  "leading_hypothesis_id": "h1",
  "evidence_quality": 0.8,
  "hypothesis_quality": 0.7,
  "progress": 0.7,
  "uncertainty_reduced": true,
  "unsupported_claims": [],
  "critique": "len([]) == 0 confirmed by execution; contract still undecided."
}
```

D. Simulation

In classical MCTS a simulation is a random playout. In LATS it is a **guided rollout with real interaction**: no learned world model is used. From the best new child the agent repeatedly (1) asks the policy for actions, (2) executes the most promising one in the environment, (3) evaluates the new state, and (4) continues until a terminal condition. The tree is extended with these nodes, so the rollout is not thrown away.

- **Terminal conditions:** a conclude action; depth limit reached; no valid actions; wall-clock or token budget exhausted.
- **Depth limit:** max_depth (6 to 8 for single-function bugs). At the limit the node is terminal with a forced low value unless the state already supports a hypothesis.
- **Rollout depth:** rollout_depth (2 to 4) caps one simulation; deeper investigation happens across iterations.
- **Budget:** a global cap on iterations, LLM calls and executions. Track them in a Budget object and stop gracefully.

```
FUNCTION simulate(node):
    cur <- node
    REPEAT up to ROLLOUT_DEPTH times:
        IF cur is terminal OR cur.depth >= MAX_DEPTH: BREAK
        kids <- expand(cur, k = 1)            // greedy: single best action, real execution
        IF kids is empty: mark cur pruned; BREAK
        cur <- kids[0]
        cur.value <- evaluate(cur)
    RETURN (cur, cur.value)
```

E. Backpropagation

After simulation, the terminal (or last) value v is pushed from that node to the root. Each node on the path updates its visit count and value sum; the average is the Q used by UCT:

```
N(s) <- N(s) + 1
W(s) <- W(s) + v
Q(s) = W(s) / N(s)          (equivalently  V_new = ((N-1)*V_old + v) / N)
```

Optionally apply a discount gamma per step toward the root (v <- gamma * v), which prefers shorter investigations. With gamma = 1 every ancestor sees the same value.

```
FUNCTION backprop(node, v, gamma = 1.0):
    WHILE node is not NULL:
        node.visits    <- node.visits + 1
        node.value_sum <- node.value_sum + v
        v              <- v * gamma             // optional discount toward the root
        node           <- node.parent
```

F. Reflection

When a rollout ends in failure (wrong conclusion refuted by evidence, depth limit with no supported hypothesis, repeated dead end), the reflection LLM receives the trajectory (actions, real observations, final state) and writes a short lesson: what was tried, why it did not help, what to try instead. It is stored on the terminal node and appended to a bounded memory list that the policy sees in later expansions. This is the LATS mechanism that lets search improve across iterations without gradient updates.

- Reflect on **failed or stalled** trajectories only, to save budget.
- Reflections must quote observations, not invent new ones.
- Keep the last 5 reflections; summarize older ones if the prompt grows too long.

5. Reward Design and Objective Evidence

The reward calculator turns facts from the environment into a number in [0,1]. It uses no LLM so it cannot be flattered by a confident explanation.

Signal	Weight	Rationale
Original error reproduced by an executed action	+0.15	Establishes a reliable observation.
New evidence item not already in state	+0.05 each (cap 0.15)	Information gain.
Hypothesis supported by an experiment	+0.20	Evidence tied to a claim.
Hypothesis refuted by an experiment	+0.10	Refutation also reduces uncertainty.
Patch based on a hypothesis makes failing tests pass	+0.30	Causal confirmation of the root cause.
Patch breaks previously passing tests	-0.20	Hypothesis too broad or wrong.
Repeated or no-information action	-0.10	Wastes budget.
Conclusion citing real evidence that matches a verified patch	terminal 0.9 - 1.0	Successful terminal.
Conclusion lacking verified evidence	terminal 0.0 - 0.2	Unsupported claim.

Because the debugging task in the example has an under-specified contract (return None or raise ValueError), a patch that adds *either* behavior counts as consistent with the test expectations ["None", "ValueError"]. The final report should still state that the contract decision is a product choice.

```
FUNCTION objective_reward(parent_state, child_state, action, obs):
    r <- 0
    IF obs.reproduced_error:            r <- r + 0.15
    new_evidence <- count(child.evidence) - count(parent.evidence)
    r <- r + min(0.15, 0.05 * max(new_evidence, 0))
    IF obs.hypothesis_supported:        r <- r + 0.20
    IF obs.hypothesis_refuted:          r <- r + 0.10
    IF obs.tests_fixed > 0:             r <- r + 0.30
    IF obs.tests_broken > 0:            r <- r - 0.20
    IF new_evidence = 0 AND action.type NOT IN {hypothesize, conclude}:
                                        r <- r - 0.10     // wasted step
    RETURN clamp(r, 0, 1)
```

6. Pseudocode: Plain Functions Only

No classes anywhere. A **state** is a dictionary, a **node** is a dictionary, and everything else is a function. Write them in this order; each function only depends on the ones above it. Notation: **x = {a, b}** is a dictionary, **x.a** reads a field, **<-** assigns.

```
Suggested files (just functions in each):
  llm_client.py   executor.py   environment.py   reward.py
  policy.py       evaluator.py  reflection.py    mcts.py   main.py
```

6.0 Settings and data shapes

```
SETTINGS: C = 1.0, K = 4, MAX_DEPTH = 7, ITERATIONS = 15, ROLLOUT_DEPTH = 3,
          LAMBDA = 0.5, BETA = 0.5, M_SAMPLES = 3, MAX_LLM_CALLS = 300, TIMEOUT = 5
reflections <- empty list           // global memory of lessons
llm_calls   <- 0                    // global counter

state = { original_code, current_code, error, tests, expected_behavior,
          test_results, hypotheses, actions_taken, evidence }
        // hypotheses item: {id, text, status}      evidence item: {id, kind, text}

node  = { id, parent, children, state, action, observation, depth,
          visits, value_sum, reward, reflection, terminal_status, pruned }
        // average value = value_sum / visits     (0 if visits = 0)
```

6.1 LLM helpers

```
FUNCTION llm_call(system, user, temperature):
    llm_calls <- llm_calls + 1
    RETURN text returned by the LLM API for (system, user, temperature)

FUNCTION llm_json(system, user, temperature):
    REPEAT 3 times:
        text <- llm_call(system + " Return ONLY valid JSON.", user, temperature)
        IF text contains valid JSON: RETURN the parsed JSON
    RETURN NULL
```

6.2 Running code safely

```
FUNCTION is_safe(snippet):
    IF snippet contains any of [import os, subprocess, socket, shutil, open(, exec(, eval(, sys.exit]:
        RETURN false
    RETURN length(snippet) <= 2000

FUNCTION run_python(source):
    run source in a separate process with a TIMEOUT-second limit
    RETURN {stdout, stderr, timed_out}

FUNCTION run_tests(code, tests):
    results <- empty list
    FOR each test IN tests:                      // test = {call, expect_list}
        out <- run_python(code + "\n" + "evaluate test.call; print RESULT value or EXC type")
        got <- result value, exception type name, or "timeout"
        passed <- (got IS IN test.expect_list)   // if no expect_list: passed = no exception
        append {call: test.call, got, passed} to results
    RETURN results
```

6.3 State and actions

```
FUNCTION make_state(code, error, tests, expected):
    RETURN {original_code: code, current_code: code, error, tests, expected_behavior: expected,
            test_results: [], hypotheses: [], actions_taken: [], evidence: []}

FUNCTION render_state(state):                    // text shown to the LLM
    RETURN code + error + expected behavior + test_results + hypotheses
           + last 10 evidence items + last 10 actions_taken

FUNCTION add_evidence(state, kind, text):
    append {id: "e" + (count(state.evidence) + 1), kind, text} to state.evidence

FUNCTION validate_action(state, action, sibling_actions):
    IF action.type NOT IN [inspect_code, run_snippet, run_tests, hypothesize, apply_patch, conclude]:
```

```
        RETURN (false, "unknown type")
    IF a required field is missing:      RETURN (false, "missing field")
          // inspect_code: focus | run_snippet: code | hypothesize: hypothesis
          // apply_patch: new_code, hypothesis_id | conclude: root_cause, evidence_ids
    IF action IN state.actions_taken OR action IN sibling_actions:
        RETURN (false, "duplicate")
    IF action has code or new_code AND NOT is_safe(that code):
        RETURN (false, "unsafe code")
    IF action.type = conclude AND any cited evidence id is not in state.evidence:
        RETURN (false, "unknown evidence")
    RETURN (true, "")

FUNCTION run_action(state, action):              // the ONLY place observations are created
    new_state <- deep copy of state
    append action to new_state.actions_taken
    obs <- {text: "", reproduced_error: false, tests_fixed: 0, tests_broken: 0,
            hypothesis_supported: false, hypothesis_refuted: false}

    IF action.type = inspect_code:
        obs.text <- source lines containing action.focus (with line numbers)
        add_evidence(new_state, "inspect", obs.text)

    IF action.type = run_snippet:
        out <- run_python(new_state.current_code + action.code)
        obs.text <- out.stdout + out.stderr
        obs.reproduced_error <- (out.stderr contains the original error type)
        add_evidence(new_state, "run", action.code + " -> " + obs.text)

    IF action.type = run_tests:
        new_state.test_results <- run_tests(new_state.current_code, new_state.tests)
        obs.text <- one PASS/FAIL line per test
        add_evidence(new_state, "tests", obs.text)

    IF action.type = hypothesize:
        append {id: "h" + number, text: action.hypothesis, status: "open"} to new_state.hypotheses
        obs.text <- "recorded, unverified"             // adds NO evidence

    IF action.type = apply_patch:
        before <- run_tests(new_state.current_code, new_state.tests)
        after  <- run_tests(action.new_code, new_state.tests)
        obs.tests_fixed  <- count of tests failing before and passing after
        obs.tests_broken <- count of tests passing before and failing after
        new_state.current_code <- action.new_code
        IF obs.tests_fixed > 0 AND obs.tests_broken = 0:
            set hypothesis action.hypothesis_id to "supported";  obs.hypothesis_supported <- true
        ELSE:
            set it to "refuted";  obs.hypothesis_refuted <- true
        obs.text <- "fixed=" + obs.tests_fixed + " broken=" + obs.tests_broken
        add_evidence(new_state, "patch", obs.text)

    IF action.type = conclude:
        obs.text <- "conclusion recorded"

    RETURN (new_state, obs)
```

6.4 Rewards

```
FUNCTION objective_reward(old_state, new_state, action, obs):
    r <- 0
    IF obs.reproduced_error:       r <- r + 0.15
    new_ev <- count(new_state.evidence) - count(old_state.evidence)
    r <- r + min(0.15, 0.05 * new_ev)
    IF obs.hypothesis_supported:   r <- r + 0.20
    IF obs.hypothesis_refuted:     r <- r + 0.10
    IF obs.tests_fixed > 0:        r <- r + 0.30
    IF obs.tests_broken > 0:       r <- r - 0.20
    IF new_ev = 0 AND action.type NOT IN [hypothesize, conclude]:  r <- r - 0.10
    RETURN clamp(r, 0, 1)

FUNCTION terminal_reward(state, action):         // for conclude actions only
    cited_patch <- any cited evidence item of kind "patch" with fixed > 0
    supported   <- any hypothesis with status "supported"
    IF supported AND cited_patch:                RETURN (0.95, "success")
    IF supported OR count(action.evidence_ids) >= 2:  RETURN (0.50, "success")
    RETURN (0.10, "failure")
```

6.5 The three LLM jobs

```
FUNCTION propose_actions(state, k, feedback):                        // POLICY
    user <- render_state(state) + last 5 reflections + "Propose k diverse actions as JSON" + feedback
    result <- llm_json(POLICY_PROMPT, user, 0.8)
    IF result is NULL: RETURN empty list
    RETURN first k items of result.actions

FUNCTION evaluate_state(state, last_obs):                            // EVALUATOR
    samples <- empty list
    REPEAT M_SAMPLES times:
        out <- llm_json(EVALUATOR_PROMPT, render_state(state) + last_obs.text, 0.8)
        IF out has a numeric score: append out to samples
    IF samples is empty: RETURN 0.3
    LM <- average of clamp(sample.score, 0, 1)
    SC <- (size of the largest group of samples sharing the same leading_hypothesis_id) / count(samples)
    RETURN LAMBDA * LM + (1 - LAMBDA) * SC

FUNCTION reflect(node):                                              // REFLECTION
    steps <- every (action, observation.text) on the path from root to node
    RETURN llm_call(REFLECTION_PROMPT, steps + render_state(node.state), 0.4)
```

6.6 The six LATS functions

```
FUNCTION make_node(parent, state, action, obs, reward):
    node <- {id: new id, parent, children: [], state, action, observation: obs,
             depth: (parent.depth + 1 if parent else 0), visits: 0, value_sum: 0,
             reward, reflection: NULL, terminal_status: NULL, pruned: false}
    IF parent: append node to parent.children
    RETURN node

FUNCTION uct(parent, child):
    IF child.visits = 0: RETURN +infinity
    RETURN child.value_sum / child.visits + C * sqrt( ln(max(parent.visits, 1)) / child.visits )

// A. SELECTION
FUNCTION select(root):
    node <- root
    WHILE node.children is not empty AND node.terminal_status is NULL:
        live <- children of node with pruned = false
        IF live is empty: node.pruned <- true;  RETURN root
        node <- the child in live with the highest uct(node, child)
    RETURN node

// B. EXPANSION
FUNCTION expand(node, k):
    made <- empty list;  feedback <- ""
    REPEAT up to 2 times:
        actions <- propose_actions(node.state, k, feedback)
        siblings <- actions of node's existing children;  rejected <- empty list
        FOR each action IN actions:
            (ok, why) <- validate_action(node.state, action, siblings)
            IF NOT ok: append why to rejected;  CONTINUE
            (new_state, obs) <- run_action(node.state, action)
            r <- objective_reward(node.state, new_state, action, obs)
            child <- make_node(node, new_state, action, obs, r)
            append action to siblings
            IF action.type = conclude:
                (child.reward, child.terminal_status) <- terminal_reward(new_state, action)
            ELSE IF child.depth >= MAX_DEPTH:
                child.terminal_status <- "depth"
            append child to made
        IF made is not empty: BREAK
        feedback <- "Rejected: " + rejected
    IF made is empty: node.pruned <- true
    RETURN made

// C. EVALUATION
FUNCTION evaluate(node):
    IF node.terminal_status IN ["success", "failure"]: RETURN node.reward
    v_llm <- evaluate_state(node.state, node.observation)
    RETURN clamp(BETA * v_llm + (1 - BETA) * node.reward, 0, 1)

// D. SIMULATION
FUNCTION simulate(node, value):
    REPEAT ROLLOUT_DEPTH times:
        IF node.terminal_status is not NULL OR node.depth >= MAX_DEPTH: BREAK
```

```
        kids <- expand(node, 1)
        IF kids is empty: BREAK
        node <- kids[0];  value <- evaluate(node)
    RETURN (node, value)

// E. BACKPROPAGATION
FUNCTION backprop(node, v):
    WHILE node is not NULL:
        node.visits <- node.visits + 1
        node.value_sum <- node.value_sum + v
        node <- node.parent
```

6.7 Main search loop and main program

```
FUNCTION search(root):
    root.visits <- 1
    REPEAT ITERATIONS times:
        IF llm_calls >= MAX_LLM_CALLS: BREAK
        leaf <- select(root)
        IF leaf.terminal_status is not NULL:
            backprop(leaf, leaf.reward);  CONTINUE
        kids <- expand(leaf, K)
        IF kids is empty:
            backprop(leaf, 0);  CONTINUE
        FOR each kid IN kids: kid.value <- evaluate(kid)
        best <- kid with the highest value
        FOR each kid IN kids except best: backprop(kid, kid.value)
        (end, v) <- simulate(best, best.value)
        backprop(end, v)
        // F. REFLECTION on failure or a weak, stalled path
        IF end.terminal_status IN ["failure", "depth"] OR (end.terminal_status is NULL AND v < 0.4):
            end.reflection <- reflect(end)
            append end.reflection to reflections
    RETURN root

FUNCTION best_conclusions(root):
    all_nodes <- every node in the tree
    concl <- nodes whose action.type = "conclude"
    RETURN concl sorted by (reward, value_sum / visits), highest first

MAIN:
    state <- make_state(code, error, tests, expected_behavior)
    root  <- make_node(NULL, state, NULL, NULL, 0)
    search(root)
    best  <- first of best_conclusions(root)
    IF none:  print "inconclusive" + highest-average-value path + evidence + open hypotheses
    ELSE:     print llm_call("Write a root cause report using ONLY the listed evidence,
                              cite evidence ids, state uncertainty, treat None-vs-ValueError
                              as a product decision",
                              best.action.root_cause + best.state.evidence + best.state.current_code, 0.2)
```

7. Prompts

7.1 prompts/policy_prompt.txt

```
You are the policy of a debugging agent. Given the current investigation state, propose
DIVERSE next debugging actions. You do not run code and you never state test results; the
environment executes actions and returns real observations.

Rules:
- Return JSON: {"reasoning": str, "actions": [ ... ]}
- Allowed action types and required fields:
  inspect_code {focus}, run_snippet {code}, run_tests {}, hypothesize {hypothesis},
  apply_patch {new_code, hypothesis_id}, conclude {root_cause, evidence_ids}
- Every action includes a short "rationale".
- Actions must differ in purpose (inspect vs probe vs hypothesize vs test). No near duplicates.
- Do not repeat anything listed under ACTIONS TAKEN.
- Use lessons from failed paths. Never contradict recorded evidence.
- Only use "conclude" when at least one hypothesis has been tested by apply_patch or by
  direct experiments, and cite the real evidence ids supporting it.
- Probe snippets must only call functions defined in the code; no file, network, or os access.
- Consider the stated expected-behavior contract, not just the crash.
```

7.2 prompts/evaluator_prompt.txt

```
You evaluate the QUALITY OF AN INVESTIGATION STATE for a debugging task, not just the code.
Judge: evidence collected, hypothesis quality, progress toward the root cause, correctness
of observations, relevance of actions, and whether uncertainty has decreased.
Evidence marked "from real execution" is ground truth. Hypotheses are unverified claims.
Penalize claims not backed by evidence. Do not reward confident wording.

Return JSON:
{"score": 0..1, "leading_hypothesis_id": str|null, "evidence_quality": 0..1,
 "hypothesis_quality": 0..1, "progress": 0..1, "uncertainty_reduced": bool,
 "unsupported_claims": [str], "critique": str}
```

7.3 prompts/reflection_prompt.txt

```
You review a debugging trajectory that failed or stalled. Using ONLY the actions and
observations shown, write 3-5 sentences: (1) what was tried, (2) why it did not advance the
root cause, (3) which hypothesis seems wrong or unproven and why, (4) what different kind of
action to try next. Quote real observations; never invent results.
```

8. Worked Example: calculate_average([])

Below is a plausible run (values are illustrative). The root is the original code with error ZeroDivisionError and 3 tests (one failing).

Iter	Selected	Expanded children (real observation)	Values	Outcome
1	root	a) run_tests: [] -> ZeroDivisionError FAIL; others PASS. b) inspect_code "return": line 6 divides by len(numbers). c) run_snippet print(len([])) -> 0. d) hypothesize h1: no empty guard.	0.58 / 0.51 / 0.55 / 0.40	simulate from (a)
2	child a	e) run_snippet calculate_average([0]) -> 0.0 (zero-sum is fine). f) apply_patch guard len==0 raise ValueError (needs h1).	0.62 / 0.70	f fixes 1 test, breaks none: h1 supported
3	h1 branch	conclude: root cause = division by len(numbers) without handling empty input; cites e2,e3,e5.	terminal 0.95	success, backprop
4	UCT	Selection revisits sibling "loop is wrong" branch (high exploration bonus): run [1,2,3] -> 2.0 PASS refutes it.	0.20	reflection: "loop logic works; look at denominator"

The final report states: root cause is *unguarded division by len(numbers) when the input is empty; the function has no defined contract for empty input*; evidence ids; and the fix options (return None vs raise ValueError) as a decision for the owner, not something the agent can verify.

9. Budgets, Depth Limits, Safety and Failure Handling

Parameter	Start value	Effect
iterations	15	Number of select-expand-simulate loops. Raise for harder bugs.
k (actions per expansion)	4	Branching factor; higher = broader search, more cost.
max_depth	7	Longest investigation path.
rollout_depth	3	Greedy steps per simulation.
c (UCT)	1.0	0.5 for focused search, 1.4 for broad exploration.
lambda	0.5	LM vs self-consistency weight.
beta	0.5	LLM value vs objective reward. Increase objective share if scores are noisy.
evaluator samples m	3	Self-consistency estimate; more samples cost more calls.
max_llm_calls	300	Hard budget; search stops gracefully.
executor timeout	5 s	Stops infinite loops in generated snippets.

Rough cost per iteration: 1 policy call + k * m evaluator calls + about 3 more for the rollout + 0 or 1 reflection call. With defaults that is roughly 20 to 30 calls per iteration, so 15 iterations is a few hundred calls. Reduce m to 2 or evaluate only the top 2 children if cost matters.

Failure handling

- **LLM returns bad JSON:** retry twice (already in complete_json); then skip the call and fall back to a neutral value (0.3) for evaluation or no actions for policy.
- **All actions invalid:** feed rejection reasons back once; then prune the node.
- **Executor timeout or crash:** becomes an observation, never an exception that kills the search.
- **No conclusion by the end:** report the highest-Q path, labeled "inconclusive", with the evidence gathered and open hypotheses.
- **Contradictory conclusions:** rank by terminal reward and cited verified evidence, not by LLM confidence.
- **Safety:** run the executor inside a container with no network; keep the forbidden-pattern checks as a first filter only.

10. Testing, Tuning and Evaluation

10.1 test_debugger (pseudocode)

```
TEST test_run_tests_reports_failure:
    code  <- "f(xs) returns sum(xs) / len(xs)"
    tests <- [ f([]) expects {None, ValueError},  f([2,4]) expects {3.0} ]
    results <- run_tests(code, tests)
    ASSERT results[0].status = exception AND results[0].passed = false
    ASSERT results[1].passed = true

TEST test_validation_rejects_forbidden_and_duplicates:
    state <- env.make_state(code, "ZeroDivisionError", tests, "")
    ASSERT validate(state, {type: run_snippet, code: "import os"}).ok = false
    state <- step(state, {type: run_tests})
    ASSERT validate(state, {type: run_tests}).ok = false      // duplicate

TEST test_patch_fixes_and_updates_hypothesis:
    state <- step(state, {type: hypothesize, hypothesis: "no guard"})
    patched <- "f raises ValueError when xs is empty, else returns sum(xs)/len(xs)"
    (state, obs) <- step(state, {type: apply_patch, new_code: patched, hypothesis_id: h1})
    ASSERT obs.tests_fixed = 1 AND state.hypotheses[0].status = supported

TEST test_search_budget_respected:
    run search with iterations = 2 and a stub LLM returning fixed JSON
    ASSERT search terminates AND llm.calls <= max_llm_calls
```

10.2 Evaluating the agent

- **Benchmark set:** 20 to 50 small buggy functions with known root causes (off-by-one, wrong operator, missing guard, mutable default argument, wrong sort key, integer vs float division, shadowed variable).
- **Metrics:** root-cause accuracy (judged against the known cause), evidence-grounding rate (fraction of report claims that cite real evidence ids), calls and executions per solved task, and iterations to first correct conclusion.
- **Baselines:** single-shot LLM debugging; ReAct without search; LATS with beta=0 (LLM-only value) vs beta=0.5 (hybrid). The ablation shows what objective evidence contributes.
- **Ablate:** no reflection, no self-consistency, c in {0.5, 1.0, 1.4}, k in {2, 4, 6}.

10.3 Debugging the debugger

- Log every node: id, parent, action, observation, value, Q, N. Print the tree after the search.
- If the same branch is picked forever, lower c is too small or values saturate near 1.0; raise c or add a visit penalty.
- If no conclusions happen, the policy prompt is too strict about evidence; lower the bar or increase max_depth.
- If the evaluator always gives about 0.8, add negative examples to the evaluator prompt and raise the objective share (lower beta).

11. Limitations and Extensions

- **Evaluator bias** is reduced, not eliminated, by objective evidence; very subtle bugs with no failing test give little objective signal.
- **Cost:** tree search multiplies LLM calls. Use caching of identical states and cheaper models for evaluation.
- **Search is stochastic:** run with a fixed temperature and seed where possible, and compare several runs.
- **Contracts are partly human decisions:** the agent can find that behavior for empty input is undefined, but cannot decide whether None or ValueError is right.
- **Extensions:** run real debuggers (pdb scripts) as another action type; mutation testing to create extra discriminating tests; parallel expansion; a learned value model trained from logged trajectories; multi-file repositories with static analysis tools as additional deterministic actions.

Checklist before you run

- Save the three prompts from section 7 as text files and load them into POLICY_PROMPT, EVALUATOR_PROMPT and REFLECTION_PROMPT.
- Install your LLM provider SDK and set its API key.
- Write the functions in section 6 in the order given; they only depend on functions above them.
- Implement and run the tests in section 10.1 first (no LLM needed), then main.
- Put the executor in a container before running on untrusted code.